

MobCast: Overlay Architecture for Seamless IP Mobility using Scalable Anycast Proxies

Christopher P. Lee, Keshav Attrey, Carlos Caballero, Nick Feamster, Milena Mihail, John A. Copeland

Schools of Electrical and Computer Engineering and Computer Science

Georgia Institute of Technology

Atlanta, Georgia 30332-0250

{chrislee@, keshav@, feamster@cc., mihail@cc., copeland@ece.}gatech.edu, ccaballero@iss.net

Abstract—We propose a routing overlay system, MobCast, for simple and efficient routing to mobile hosts. Mobcast nodes advertise the same address space at each proxy location, and each mobile host is assigned a “universal” IP address from this address space, so packets sent to a mobile host’s universal IP address automatically go to the nearest proxy on the overlay. The overlay then delivers the packets to the mobile host.

Our architecture enables seamless mobility for both micro and macro mobility. While our initial design is not as mature as Mobile IP, it shows great promise to solve the traditional problems of ingress routing, firewalls, NATs, and rapid mobility with much lower complexity. We present our design as a scalable and deployable alternative to Mobile IP.

In this paper, we focus on describing the MobCast system architecture. We form our arguments for scalability, handoff-speed, and simplicity, and give our initial results for scalability. We postpone a detailed discussion of MobCast’s security model for future work.

I. INTRODUCTION

Ubiquitous wireless access has changed the way we live and work, but most common internet applications, such as email and web browsing, would not benefit from seamless mobility. A person can travel between wireless access points at different coffee shops, changing IP addresses each time, and still access his email at each one. However, seamlessness is required when using time-sensitive media streaming applications such as voice over IP (VoIP) and video conferencing. As technology enables seamlessness for these application, other applications will arise that utilize this technology. The basic problem of seamless mobility is that a TCP connection is not defined between hosts but between IP addresses that identify points of connectivity to the Internet, and changing IP addresses breaks active TCP connections.

A. Mobility Analogy

Let’s provide an analogy using the postal mail system. Bob’s firm is in Atlanta, but he has customers internationally that correspond with him through the postal service. Without constantly updating his customers on his address, how can Bob exchange mail with his customers efficiently while traveling?

Mobile IP’s solution is to have Bob’s customers (the corresponding nodes) to mail him at his Atlanta address, where his firm (his home agent) receives his mail and forwards it to his current location (his care-of address). Bob simply has to keep his firm up-to-date on his whereabouts.

A frequent problem is that when Bob is in Hong Kong, his customers in Hong Kong mail him via his firm in Atlanta, which takes too long. Bob would like to mail back these customers directly, but then he must use a Hong Kong “from” address rather than his permanent Atlanta address, or else the Hong Kong postal system will reject the mail (ingress filtering). If Bob’s customers receive mail from a Hong Kong address (instead of his Atlanta address), they cannot be sure it is really him. But he does not want to mail his customers via his Atlanta office.

Our solution addresses these problems. With MobCast, Bob’s firm purchases a permanent forwarding address for Bob at International Mail, a postal mail service with branches around the world. Bob supplies all his customers with his permanent forwarding address, e.g., “Member #525, International Mail,” and he informs International Mail whenever he moves.

His customers can reach him by sending mail to their nearest International Mail branch, which forwards the mail to Bob, wherever he is. So when Bob is in Hong Kong, mail from Bob’s Hong Kong customers reach him via the Hong Kong branch of International Mail. When Bob replies to them, he also sends his mail via the Hong Kong branch of International Mail. Since the mail comes from “International Mail,” his customers know that it is still the same person.

II. RELATED WORK

Mobile IP [7] is the foremost solution for enabling seamless mobility without altering the protocol stack. The mobile node associates with its home agent and uses an address allocated by the home agent to form connections. When the mobile node moves to a foreign network, it needs a care-of address to continue communications. A foreign agent forms a tunnel with the home agent and decapsulates the packets before delivering them to the mobile node, as seen in Figure 1. Packets from the corresponding node to the mobile node must always pass through the home agent. Packets in the opposite direction, from the mobile node to the corresponding node, can be sent directly without passing through the home agent (*triangle routing*).

Mobile IP is not without its limitations. First, all mobile nodes must have a home agent [7]. The current network must act as a home agent for them, or they must use a pre-designated home agent from another network. Furthermore,

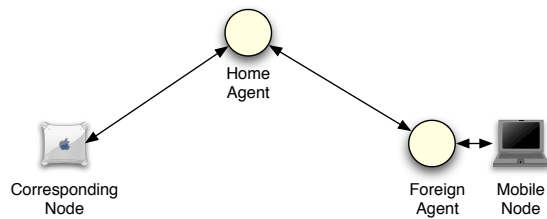


Fig. 1. A Mobile IP topology with the home agent forwarding packets to a foreign agent to reach the mobile node

networks that are close geographically may be distant in the network topology, making triangle routing unscalable and expensive in terms of latency. Second, ingress filtering can prevent packets from flowing directly from the mobile host to the corresponding node, because the mobile node must use its home address as the source. One solution to this is *reverse tunneling* [5], which sends a packet back to the home agent before forwarding it on to the corresponding node. Lastly, moves must be authenticated to prevent denial of service or connection hijacking, but the latency and signaling overhead make seamless mobility difficult and costly. While macro-mobility is handled by the Mobile IP protocol, micro-mobility extensions have been proposed to allow a mobile node to migrate its registration quickly within larger networks, allowing faster switch-overs and less signaling overhead. Reinbold [8] compared several micro-mobility protocols – Hierarchical Mobile IP, Fast Handoff, Proactive Handoff, TeleMIP, Cellular IP, HAWAII, and EMA – each of which requires additional infrastructure.

Resilient Overlay Networks (RONs) [1] form an overlay network over the existing internet routing topology. Through active monitoring, a RON node detects internet failures and routes around them by forwarding packets through another RON node before delivering it to the final destination. The tradeoff to note is that while BGP scales, it converges slowly; conversely, RON converges quickly, but does not scale. Since RON only needs a small number of nodes (tens) to operate, scalability is not a primary concern. RON can efficiently route around BGP routing failures and provide much more reliable service.

Resilient Overlay Networks have been used to form a distributed home agent for Mobile IP, allowing for route optimization. RON can find the optimal entry and exit points of the overlay for packets to be routed from the corresponding node to the mobile node. Since the RON is a distributed home agent, the trust model of Mobile IP still holds. This would require that the overlay is controlled by and trusted by an organization and would not allow for ubiquitous usage for mobile users at large.

Global IP-Anycast (GIA) [4] decreases the number of advertised routes by dividing inter-domain anycast routing into two components. One component builds inexpensive default anycast routes and the other generates enhanced anycast routes that are custom to the domain. GIA uses caching to store the enhanced routes that are popular with the users of a domain.

This requires modifications to the routers of the domain, but allows anycast services to scale without burdening the internet infrastructure. While this does reduce the load on routers, it was not intended for mobility and can work against mobility due to caching.

The *Internet Indirection Infrastructure (i3)* [10] is a highly generalized architecture that uses an overlay network to forward tagged packets to interested nodes. Using this generic architecture, i3 can enable anycast, multicast, mobility, peer-to-peer networking, and normal unicast-style connections. This requires modifications to each node sending or receiving packets on the network—specifically, a new layer inserted below the transport layer to tag the packets and send them to a rendezvous node.

These current solutions can not enable seamless mobility without altering the protocol stack or without requiring each organization to deploy their own agents. Our solution, MobCast, allows for seamless mobility in a scalable fashion, requiring only a small number of organization-agnostic, routing overlay nodes that could be used by any mobile user. We believe that ASes would want to setup anycast proxies, both as a service to their customers (to make money) and to reduce bandwidth utilization to other ASes (to save money).

III. ANYCASTING AND ANYCAST PROXIES

In 1993, Partridge et al. proposed anycast [6] as a way of duplicating a service without requiring the discovery of the new internet address, by having each target server advertise the same anycast address. For a client to locate a service, it performs the normal domain name server (DNS) lookup to resolve a host name to an internet address. The DNS returns a normal record with an anycast address as the internet address. The client then forms a connection to the service. Internet routing handles the delivery of the packets to the closest anycast server matching the anycast address; typically, this is the server topologically nearest the client. In Figure 2, each client connects to the target anycast server located closest to itself.

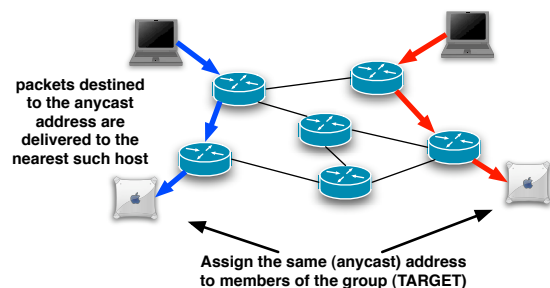


Fig. 2. Example topology using anycast. Each client communicates with the closest target server.

Anycast wastes the limited IPv4 space, because each “service” must advertise a separate address block of adequate size (/22 or larger). Ballani et al. proposed Proxy IP Anycast Service (PIAS) in [2] to solve the scalability problem and perform load balancing by having the anycast proxies form an

overlay network in which all nodes advertise the same address block. Clients, instead of connecting directly to the service they desire, use the anycast address to connect to the closest anycast proxy. The overlay system then forwards packets to the service. This is illustrated in Figure 3. The PIAS system uses port numbers as service identifiers.

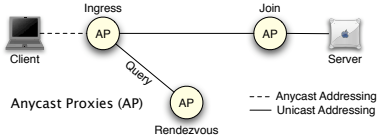


Fig. 3. A simple connection between client and server using PIAS

To enable *scalable, seamless* mobility, the MobCast architecture implements PIAS-style anycast proxies that allow quick reregistering of mobile nodes, buffering of packets in flight, and forwarding of packets from the previously connected anycast proxy to the new anycast proxy. This forms a simple, deployable, optimal routing, highly scalable, seamless mobility overlay architecture that requires no changes to any protocol stacks and makes no assumptions about the care-of addresses.

IV. MOBCAST ARCHITECTURE

MobCast extends PIAS [3] by adding mobility extensions to the anycast proxy architecture. Every proxy advertises the same subnet (/22). When a client node wants to access a service, it will use one of these anycast addresses. The packets will reach the nearest proxy (ingress proxy) and the proxy forwards the packets to the anycast proxy (join proxy) nearest to the service. The ingress proxy forwards the packets using IP encapsulation. The join proxy then delivers the packets to the service. The service relays the packets back to the join proxy that in turn forwards the packets back to the client.

For this system to scale, the anycast address and protocol port numbers identify the service desired. This allows a subnet of 1024 hosts (/22), each with 2^{16} ports, to map to 67 million services. When that begins to run out, another allotment can be advertised for the same proxies. MobCast enables seamless mobility by forwarding packets in-flight from the old join proxy to the new join proxy and by creating a tunnel between the mobile node and the join proxy to work through network address translation (NAT) firewalls.

A. A Connection Example

Following is a scenario of a mobile node (MN) registering with the anycast proxy network, being assigned an anycast address, requesting and receiving data from a corresponding node CN via the proxy network, moving to a different location where it reregisters with the anycast proxy network, and continuing to receive data at its new location.

1) *Registering to the Join Proxy:* The mobile node (MN) creates a tunnel interface device with a virtual private IP address (e.g., 10.0.13.24). This interface is used to maintain a consistent protocol stack. Next, the MN creates a connection

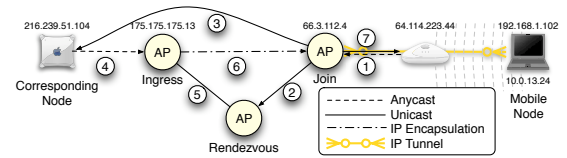


Fig. 4. MobCast MN registration and forming a connection to a CN, and the CN sending packets back.

to the join proxy using IP tunneling (connection 1 in Figure 4). The destination address is the anycast address of the proxy, so that the closest proxy will be contacted. This tunnel also has the benefit of creating a return path through a NAT, if present. This tunnel will be maintained throughout the time that the mobile node is connected via the network. Then, a registration message containing the node's certificate is sent to the join proxy. The join proxy will verify the registration and contact the set of rendezvous proxies corresponding to this service, which will also verify the registration (connection 2 in Figure 4). The join proxy will send a token, $\langle \text{Time}, \text{anycast proxy ID}, \text{MN ID} \rangle$, back to the mobile node via the IP tunnel. This token is used to authenticate a switch-over to a new join proxy.

2) *Initiating Connections:* When the mobile node (MN) initiates a connection to a corresponding node (CN), (e.g., 216.239.51.104), the packet is sent via the tunneling device to the join proxy. The join proxy then sends the packet directly to the CN with the anycast address as the source IP and the source port set to a port that maps back to the MN (connection 3 in Figure 4). The reply from the CN is addressed to the anycast address and goes to the nearest proxy (ingress proxy) (connection 4 in Figure 4). The ingress proxy does not know which proxy is the join proxy for the MN. It determines the list of rendezvous proxies for the MN and forwards the packet to the nearest one. The rendezvous proxy notifies the ingress proxy of the MN's join proxy, and forwards the packet to the join proxy. After being notified of the join proxy, the ingress proxy forwards packets directly to the join proxy. The join proxy delivers the packets to the MN via the tunnel.

3) *Migration:* When the mobile node switches networks, it receives an internet address from the new network and will have to perform a switch over. The switch over involves reconnecting to a join proxy. If the MN reconnects to the same join proxy, nothing else is needed—incoming packets will simply be forwarded through the new, reestablished tunnel as seen in Figure 5. However, if it connects to a different join proxy, it registers with the new proxy. The new join proxy informs both the old join proxy and the rendezvous proxies of the transition, using the token for authentication (Figure 6). The old join proxy will then contact the rendezvous proxies to de-register itself from serving the MN. Additionally, any packets in-flight to the old join proxy will be forwarded to the new join proxy.

Since the MN will likely be disconnected from the old join proxy during switch over, the old join proxy will need to buffer packets. When the new join proxy informs the old join proxy

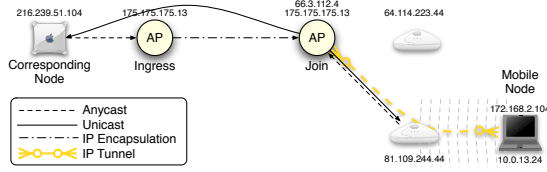


Fig. 5. MobCast MN switch over and reconnecting to the same join proxy

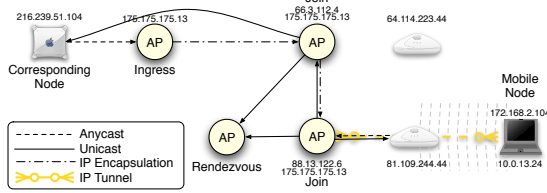


Fig. 6. MobCast MN switch over and reconnecting to a new join proxy

that it now has the MN, the buffered packets can be sent to the new join proxy. The time limit of buffering, and thus its size and scalability, is an open question, but several implementation details allow this to scale quite well. Since the join proxy uses an IP tunnel to the MN, it automatically buffers packets that have not been acknowledged and backs off if a packet times out. Once a packet is acknowledged, it can be removed from the buffer. Since the join proxy is close to the MN, the round trip time (RTT) should be short and keep the buffer quite small.

To support multiple connections that originate from the mobile node to the same corresponding node, multiple ports are needed to distinguish between the different connections. Some browsers initiate three connections for each request and after receiving the first response, kill the other two connections with a reset packet. When a MN connects to a corresponding node, the join proxy calculates an ephemeral port that maps back uniquely to that join proxy. We are still discussing the best way to allocate ephemeral ports, but the general idea is to have a range of ports or anycast addresses reserved for ephemeral connections. When a connection comes in for one of these ports/addresses, the ingress proxy will send it directly to the matching join proxy. When the MN migrates, an update message is sent to the ingress proxy to override the default join proxy.

V. SCALABILITY

To show that the mobcast system is scalable for use by both content providers and mobile users, we simulate the convergence time of host mobility (switch-off) and the memory required per mobile node on the anycast proxies. Scalability is the primary factor under investigation due to its impact on the deployability in a real world system.

A. MobCast Service Table Memory Requirements

There are two sources of memory utilization in MobCast, MobCast Service Records (MSRs) and packet buffers. On the join proxy, there is a destination service record (DSR) for each mobile node that is connecting through it. The DSR

TABLE I
EQUATION VARIABLE DEFINITIONS KEY

N	Number of services supported by the anycast proxy system
M	Modulus for assign the rendezvous proxy
P	Number of anycast proxies
B_{peak}	Maximum join proxy buffer size per mobile node
B_{avg}	Normal network buffer size in steady-state
R	Steady-state flow throughput between a MN and a CN
τ_r	Service registration latency
τ_i	Maximum interface switch-over latency
τ_{o-n}	Latency between the old join proxy and the new join proxy
τ_{rr}	Reregistration latency
τ_{RRT}	Round trip time between the join proxy and the MN

contains the anycast address and the port number that map to this mobile node, the mobile node's tunnel ID, and the security token. The size of these records is 32 bytes each after memory-word alignment is considered. On the average, a rendezvous proxy would keep $\frac{N}{M}$ join proxy service records (JSR). Please refer to Table I for variable definitions. For 67 million services, and a modulus of 14, a rendezvous proxy would store an average of 154 MB of records. If the anycast proxy is both the join proxy and the rendezvous proxy for the same service, then it would only store the destination record and thus would not duplicate the join proxy pointer record for itself. If evenly distributed, a join proxy would service $\frac{N}{P}$ number of hosts with an average overlap of $(\frac{1}{M})(\frac{1}{P})$. So the total memory needed to store the MobCast Service Table (MST) would be: $20 \times N \times [\frac{1}{M} + \frac{1}{P} - \frac{1}{M} \cdot \frac{1}{P}]$ bytes

For 67 million services and only 50 anycast proxies, a join proxy would service 1.3 million mobile nodes. The average overlap in this situation would be about 0.14% or 96 thousand records (a total of 3MB). The total memory requirement for 67 million services on each anycast proxy will average: 32 bytes $\times$ 67E6 records $\times$ ((1/14)+(1/50)-((1/14)*(1/50))) = 184.32MB.

B. Network Buffering Memory Requirements

To measure the network memory utilization, we track the unacknowledged packets at the old join proxy as the mobile node moves to a new network. The upper bound is given by the steady-state data rate times the delay between the time the mobile node disconnects from the old join proxy and the time the old join proxy is informed of the new join proxy: $B_{peak} = R \times \tau_{rr}$. The time required to inform the old join proxy of the new join proxy is the time for the MN to switch over to the new network, the MN's reregistration to reach the new join proxy, and the new join proxy to send a packet to the old join proxy: $\tau_{rr} = \tau_i + \tau_r + \tau_{o-n}$.

The latency between old and new join proxies is typically small because they are near to each other, hence the reregistration time is expected to be small and thus the interface switch-over time dominates the wireless switchover. In typical situations, the mobile node would detect a degrading channel, proactively switch to a better channel at the most opportune time, and eliminate the "cold" interface switch-over latency. In simulation, the peak buffer size on the join proxy experiencing a 10 Mbps TCP flow to a mobile node was an average of

65 packets, amounting to 104 KB of buffering. We had no switch-over latency, but a cold switchover was forced every two seconds. A 100 ms interface switch-over time would add 165KB to the buffering, quickly dominating the latency associated with registration. Fast switch-over improvements will reduce this latency and decrease the pressure on the network buffers.

In the normal case, the network buffer is only the incoming rate times the round trip time from the join proxy to the MN: $B_{avg} = R \times \tau_{RTT}$. This is analogous to a normal TCP buffer except that the endpoints of the connection, the join proxy and the MN, are close to each other, while the incoming data rate is a function of the end-to-end, CN to MN, throughput. Hence, the average buffer for a 10 Mbps flow and a RTT of 5 ms would be 6.3 KB.

VI. SIMULATION RESULTS

To prove the functionality of our mobility extensions and to show that our solution scales to many mobile users, we implemented MobCast in the Georgia Tech Network Simulator (GTNetS) [9]. We simulated MobCast and measured the time required for service convergence and the network buffer sizes. The service convergence time (SCT) measures the time required for a server or mobile node to detect a failed interface, for it to send a re-registration message to a new join proxy, and for the service update message to reach the old join proxy. We define SCT this way because it is directly used to calculate the buffer required on the join proxy (per seamless mobility enabled service).

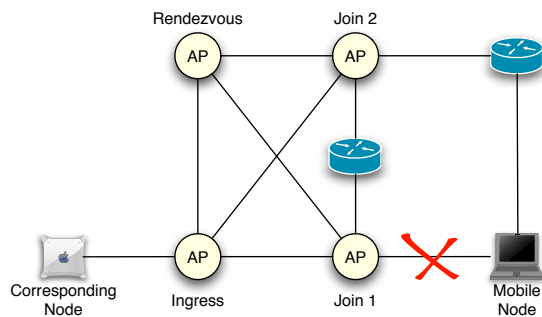


Fig. 7. MobCast MN switch over and reconnecting to a new join proxy

The first experiment, seen in Figure 7, used a full-rate TCP sender from a corresponding node to the mobile node. The links are 10 Mbps each with 5 ms of delay. The SCT for this topology was 20 ms: 10 ms to reach the new join proxy and an additional 10 ms to reach the old join proxy. Every 4 seconds, after TCP has regained steady state, we break the currently-used link and force the mobile node to switch to its other interface. We ran this simulation for a simulation time of 20 hours (72,000 seconds) and measured the peak buffer sizes of each join proxy. The average peak on join proxy 1 was 65 packets (104KB), seen in Figure 8, and the average peak on join proxy 2 was 49 packets (78KB). Further experiments with larger topologies showed a linear growth in buffer requirements with the number of flows as expected.

Even this linear growth can be slowed by having the join proxy proactively reduce the window size on behalf of the transmission, reducing the incoming rate.

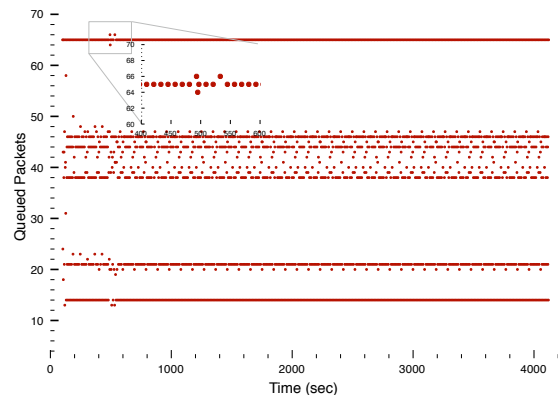


Fig. 8. Buffered packets over time with periodic switchover every 2 seconds

VII. CONCLUSION

MobCast allows simple, scalable, seamless mobility with a minimal amount of network buffering, without modifying the protocol stack or requiring massive deployments. Anycast proxies allow replicated services to be located without the scaling problems associated with Anycasting. We calculated the maximum storage required at each proxy for service records and measured in simulation the network buffer size. We further show that service convergence occurs in $O(1)$ time versus the number of services and proxies. MobCast addresses many of the original deployment issues of Mobile IP, but can still benefit from much of the research on fast-switchover and authentication methods.

REFERENCES

- [1] David G. Andersen, Hari Balakrishnan, M. Frans Kaashoek, and Robert Morris. Resilient overlay networks. In *Symposium on Operating Systems Principles*, pages 131–145, 2001.
- [2] Hitesh Ballani and Paul Francis. Towards a deployable ip anycast service. In *First Workshop on Real, Large Distributed Systems (WORLDS '04)*, Dec 2004.
- [3] Hitesh Ballani and Paul Francis. Towards a global ip anycast service. *SIGCOMM Comput. Commun. Rev.*, 35(4):301–312, 2005.
- [4] Dina Katabi and John Wroclawski. A framework for scalable global IP-anycast (GIA). In *SIGCOMM*, pages 3–15, 2000.
- [5] G. Montenegro. Reverse tunneling for mobile ip, revised, 2001.
- [6] C. Partridge, T. Mendez, and W. Milliken. Host anycasting service. RFC 1546, November 1993.
- [7] C. E. Perkins. Ip mobility support. RFC 2002, Oct 1996.
- [8] P. Reinbold and O. Bonaventure. A survey of ip micro-mobility protocols, 2002.
- [9] George F. Riley. The georgia tech network simulator. In *MoMeTools '03: Proceedings of the ACM SIGCOMM workshop on Models, methods and tools for reproducible network research*, pages 5–12, New York, NY, USA, 2003. ACM Press.
- [10] Ion Stoica, Daniel Adkins, Shelley Zhuang, Scott Shenker, and Sonesh Surana. Internet indirection infrastructure. *IEEE/ACM Trans. Netw.*, 12(2):205–218, 2004.